

Guía Oficial del Equipo

Sistema de Sincronización Canvas LMS ↔ Microsoft Teams — USIL Paraguay

Este documento es la referencia oficial para trabajar sobre el repositorio del proyecto. Explica qué es la plataforma, cómo está construida, cómo levantarla en tu máquina, sobre qué rama trabajar y qué falta para cerrar el **Avance 2** de Programación IV. Léelo antes de tocar el código.

Control del documento

Campo	Valor
Versión	1.0
Fecha	29 de septiembre de 2026
Autor	Roque Esteche — Scrum Master / Project Manager
Asignatura	Programación IV — Prof. Marcos Arrieta
Grupo	N.º 3 — Canvas LMS / Microsoft Teams

Datos del repositorio

Campo	Valor
Repositorio (GitHub)	https://github.com/RoqueLuis07/Canvas_for_Team
Rama oficial de trabajo del equipo	php_proyecto_integrador — ver en GitHub
URL de clonado (HTTPS)	<code>https://github.com/RoqueLuis07/Canvas_for_Team.git</code>
Carpeta del proyecto Laravel	<code>sistema-academico/</code>
Ramas que NO deben tocarse	<code>main, claude/webpage-sections-sync-8m633b</code>
Panel administrativo (local)	<code>http://localhost:8000/admin</code>
Login de prueba (seed)	<code>admin@usil.edu.py / password</code>

Regla número uno: todo el trabajo del equipo se hace **exclusivamente** sobre la rama **php_proyecto_integrador**. Nunca se pushea ni se hace merge hacia otra rama del repositorio. Ver sección 4.

1. ¿Qué es esta plataforma?

USIL Paraguay administra dos plataformas académicas que hoy funcionan de forma aislada: **Canvas LMS** (cursos, matrículas, calificaciones) y **Microsoft Teams / 365** (identidad institucional, correo y espacios de colaboración por curso). El personal de TI ejecuta manualmente, curso por curso y usuario por usuario, el alta de cuentas, la creación de equipos de Teams, la matriculación combinada y las bajas — un proceso lento y propenso a inconsistencias.

Esta plataforma es un **panel administrativo web** que centraliza y automatiza ese proceso: el administrador de TI crea un curso en Canvas, el sistema crea automáticamente el equipo de Teams asociado, matricula al usuario en ambas plataformas con una sola acción, y deja un registro de auditoría de cada operación.

Flujo principal (matriculación combinada)

- El administrador busca un curso sincronizado desde Canvas en el panel.
- Si el curso aún no tiene equipo de Teams, el sistema lo crea vía Microsoft Graph API.
- El administrador busca al usuario a matricular (docente o alumno).
- El sistema valida que no esté ya matriculado (evita duplicados).
- El sistema lo matricula en Canvas y lo agrega como miembro del Team correspondiente.
- Queda registrada la operación en el historial de auditoría (éxito o error, por plataforma).

2. Alcance oficial (Avance 1 ya entregado)

El Informe de Avance 1, ya presentado y calificado, define el alcance formal del proyecto. **Cualquier funcionalidad nueva debe respetar estos límites** — si tenés dudas sobre si algo entra o no, preguntá antes de programarlo.

Incluido este semestre

- Autenticación restringida a personal administrativo de TI.
- Listado y filtro de cursos sincronizados desde Canvas.
- Creación de equipos de Teams asociados a un curso (Microsoft Graph API).
- Matriculación combinada (curso + Team) en una sola acción.
- Baja combinada de usuario en ambas plataformas.
- Registro de auditoría de todas las operaciones anteriores.
- API REST documentada para las operaciones de sincronización.
- Despliegue en contenedores Docker gestionados con Portainer.
- Balanceo de carga con HAProxy entre dos arquitecturas (A y B).

Fuera de alcance este semestre

- Sincronización de calificaciones o contenido académico entre Canvas y Teams.
- Autoservicio: los estudiantes o docentes NO se auto-matriculan — el flujo siempre lo opera TI.
- Integración con el sistema de gestión académica interno de USIL más allá del código SIS del curso.
- Aplicación móvil nativa.

- Facturación o módulos administrativos ajenos a la sincronización Canvas–Teams.

Nota importante: el código evolucionó en paralelo con módulos más amplios (postulación, planes de estudio, materias, prerequisites, autoservicio de matriculación) que exceden este alcance. Se decidió realinear el código al MER y al alcance de este Avance 1 — ese trabajo está en curso. Si ves entidades como Postulacion, PlanEstudio o Materia en el código, son parte de una versión anterior más ambiciosa que se está acotando; ningún desarrollo nuevo debería depender de ellas.

3. Arquitectura técnica

- **Framework:** Laravel 13 (PHP 8.4), base de datos SQLite en desarrollo local.
- **Panel administrativo:** Filament 3.3 (/admin).
- **Integraciones externas:** Canvas API (REST) y Microsoft Graph API (Teams / Azure AD), vía Guzzle/Http de Laravel.
- **Colas y caché:** preparado para Redis (Avance 3), hoy sesiones centralizadas en base de datos.

Patrón Contract + Null-Object (clave para trabajar sin credenciales)

Las integraciones con Canvas y Teams están detrás de interfaces (App\Contracts\CanvasClient y TeamsClient). Cuando no hay credenciales reales configuradas, AppServiceProvider inyecta automáticamente una implementación simulada (NullCanvasClient / NullTeamsClient) que registra la operación en el log pero no llama a ninguna API externa real.

Esto significa que **todo el sistema funciona de punta a punta sin que nadie del equipo necesite un token de Canvas ni credenciales de Azure** — ver sección 6.

Servicios principales (App\Services)

Servicio	Responsabilidad
CursoService	Crea el curso en Canvas, crea el equipo en Teams, asigna al docente titular.
InscripcionMateriaService	Matricula, da de baja o cambia a un alumno de curso, con alta/baja automática en Canvas y Teams.
MaterialService	Publica contenido de un curso como página en Canvas.
SincronizacionService	Envuelve cada operación real contra Canvas/Teams: registra un JobSincronizacion (pendiente → en_proceso → completado/error) y un LogAuditoria.

Modelo de datos actual

Entidad	Qué representa
Persona	Cualquier actor: aspirante, alumno, docente o administrativo.
Curso	Oferta concreta de una materia en un período — lo que se crea en Canvas/Teams.
InscripcionMateria	Matrícula de una Persona en un Curso (con alta/baja en Canvas/Teams).
Material	Contenido académico de un curso, publicable como página de Canvas.
JobSincronizacion	Ciclo de vida de cada operación de sincronización disparada.
LogAuditoria	Historial de auditoría: quién hizo qué, sobre qué entidad, cuándo.

Como se indica en la sección 2, este modelo se está acotando hacia el MER exacto del Avance 1 (ROL, USUARIO, SUBCUENTA, CURSO, MATRICULA, EQUIPO_TEAMS, MIEMBRO_EQUIPO, JOB_SINCRONIZACION, LOG_AUDITORIA). Consultá con Roque antes de crear una migración nueva para evitar pisar ese trabajo.

4. Reglas de trabajo del equipo (leer antes de programar)

Rama de trabajo

Todo el equipo trabaja exclusivamente sobre la rama `php_proyecto_integrador`. Tenés total libertad para hacer commits y push directo ahí. Enlace directo a la rama: https://github.com/RoqueLuis07/Canvas_for_Team/tree/php_proyecto_integrador.

Nunca pushees ni hagas merge hacia `main` ni hacia `claude/webpage-sections-sync-8m633b` — es una rama de otro proyecto del mismo repositorio y no debe tocarse.

Flujo sugerido por tarea

- Actualizá tu copia local: `git checkout php_proyecto_integrador && git pull`.
- Antes de tocar código PHP modificado, corré `vendor/bin/pint --dirty --format agent`.
- Corré los tests relacionados: `php artisan test --compact` (toda la suite tarda ~2s).
- Commiteá con mensajes claros y hacé push directo a `php_proyecto_integrador`.
- Si dos personas tocan lo mismo al mismo tiempo, coordinen por el tablero Trello antes de empezar.

Qué nunca se commitea

- El archivo `.env` (credenciales locales) — ya está en `.gitignore`.
- `database/database.sqlite` — se genera local, no se versiona.
- Cualquier token, contraseña o secreto pegado directamente en el código.

5. Cómo levantar el proyecto en tu máquina

Requisitos: PHP 8.4, Composer, y opcionalmente Node si vas a tocar frontend. No se necesita Docker para desarrollo local.

```
git clone https://github.com/RoqueLuis07/Canvas_for_Team.git
cd Canvas_for_Team/sistema-academico
git checkout php_proyecto_integrador
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
php artisan serve
```

Con eso ya tenés el panel corriendo en `http://localhost:8000/admin`. El seeder crea un login de prueba: `admin@usil.edu.py` / `password`.

Para correr los tests: `php artisan test --compact`. Antes de cada commit con cambios PHP: `vendor/bin/pint --dirty --format agent`.

6. Seguridad y credenciales — qué necesitás y qué no

No hay ningún secreto real commiteado en el repositorio (se verificó el historial completo de la rama). Y gracias al patrón Null-Object de la sección 3, no necesitás pedirle a Roque ningún token de Canvas ni credenciales de Azure para desarrollar: dejá esas variables vacías en tu `.env` y el sistema usa automáticamente los clientes simulados.

Variables que sí necesitás (y son locales, no secretas)

- `APP_KEY` — la genera cada uno con `php artisan key:generate`, no se comparte entre personas.
- `DB_CONNECTION=sqlite` — ya viene así en `.env.example`, no hay que tocar nada.

Variables que quedan vacías salvo que se trabaje contra Canvas/Teams reales

- `CANVAS_ACCESS_TOKEN`, `CANVAS_ACCOUNT_ID`
- `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`

Si en algún momento el equipo necesita probar contra el Canvas/Azure reales de USIL, esas credenciales las administra Roque — pedir las por un canal privado (nunca por el chat del grupo ni pegadas en un commit).

7. Estado frente al calendario de la cátedra

Hito	Fecha	Estado
Avance 1	28/08	Entregado.
Avance 2	25/09 (vencido)	CRUD, sesiones centralizadas y auditoría OK. Falta Portainer y reporte de sprints.
Avance 3	30/10	Pendiente: HAProxy, dos arquitecturas activas, demo de failover.
Avance 4	27/11	Pendiente: benchmarking, dashboard HAProxy, documentación final.

8. Qué falta para cerrar Avance 2 — por rol

Rol (según Avance 1)	Tareas pendientes
Scrum Master / PM	Actualizar el tablero Trello con el Sprint 3 (HU-05/HU-06) y dejar evidencia de las ceremonias en /docs/ceremonias.
Backend & Architect Lead	Acompañar la realineación del modelo de datos al MER de Avance 1 (Persona→Usuario/Rol, Subcuenta, EquipoTeams, MiembroEquipo).
DevOps & Cloud Engineer	Importar el stack de docker-compose.yml en Portainer y dejarlo corriendo — es el único entregable de Avance 2 que falta.
Frontend & QA Engineer	Probar el flujo completo en /admin (crear curso → crear team → matricular → baja) y reportar bugs o UX confuso.

El único bloqueante real para cerrar Avance 2 es el despliegue en Portainer — es infraestructura, no requiere credenciales de Canvas/Teams.

9. Checklist antes de avisar que una tarea está lista

- Corriste `php artisan test --compact` y pasó todo.
- Corriste `vendor/bin/pint --dirty --format agent` sobre los archivos PHP que tocaste.
- Hiciste push a `php_proyecto_integrador` (nunca a otra rama).
- Actualizaste la tarjeta correspondiente en el tablero Trello.

Ante cualquier duda de alcance o de arquitectura, consultá con Roque antes de avanzar. Repositorio:
https://github.com/RoqueLuis07/Canvas_for_Team · Rama de trabajo: [php_proyecto_integrador](#)